Internet of Things: estación meteorológica

Parte I · Kafka, Productor y Consumidor

Nicolle Gordillo – 22246
Giovanni Santos – 22523
CC3067 Redes · Universidad del Valle de Guatemala

Arquitectura de la solución

Productor

- Simula temperatura, humedad y viento
- Normal con media 55 y σ 15
- Envía cada 15–30 s, key sensor1

→

Kafka (Edge)

- 161.35.11.141:9092
- SASL_PLAINTEXT / PLAIN
- Topics 22246 y 22246-compact

→

Consumidor

- group_id 22246-consumer, earliest
- Gráfica en vivo con matplotlib
- Temperatura, humedad y viento

Parte 3.1: formato JSON

¿A qué capa OSI pertenece JSON/SOAP?

- Capa 7 (Aplicación): lo producen y lo interpretan las aplicaciones
- Su función es de representación de datos, propia de la capa 6 (Presentación), que TCP/IP integra en Aplicación

¿Qué beneficios tiene?

- Modularidad: campos con nombre, sin depender del orden
- Legible para personas y de fácil depuración
- Independiente del lenguaje, con soporte en todas partes
- Menos frágil: se pueden agregar campos sin romper el consumidor

Evidencia 3.1: generación de datos

producer.py · generar_data()

Simula los tres sensores en cada lectura

```python
def generar_data() -> dict:
    """Lee los tres 'sensores' simulados."""
    temperatura = round(min(max(random.gauss(MEDIA, DESVIACION), 0), 110), 2)
    humedad = int(round(min(max(random.gauss(MEDIA, DESVIACION), 0), 100)))
    direccion = random.choice(DIRECCIONES)
    return {
        "temperatura": temperatura,
        "humedad": humedad,
        "direccion_viento": direccion,
    }
```

Ejemplo de lectura enviada (21:22:23)

```
{"temperatura": 36.45, "humedad": 35, "direccion_viento": "N"}
```

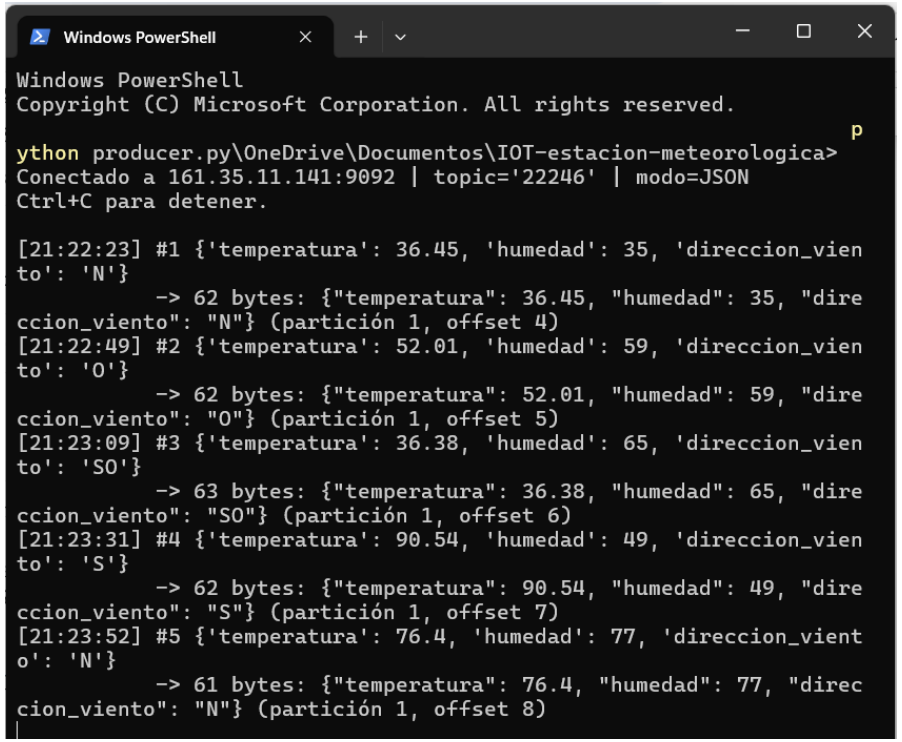
Cómo se generan

- **Temperatura:** normal con media 55 y σ = 15, recortada a [0, 110] y con 2 decimales
- **Humedad:** la misma normal, recortada a [0, 100] y convertida a entero
- **Viento:** random.choice, uniforme entre las 8 direcciones
- **¿Por qué σ = 15?** el 99.7 % de los valores cae entre 10 y 100, así que el recorte casi nunca actúa

Evidencia 3.2: envío al servidor Edge

python producer.py

Topic 22246 · key sensor1 · JSON en UTF-8



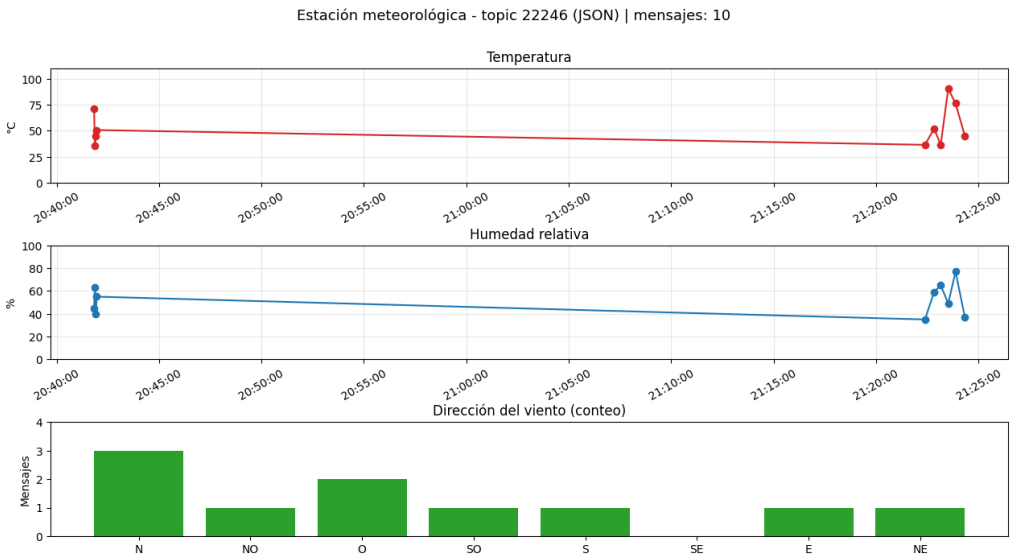
Qué muestra la captura

- Se conecta a 161.35.11.141:9092 con SASL_PLAINTEXT / PLAIN
- Cada lectura se envía como un JSON de 61 a 63 bytes
- Entre envíos pasan de 20 a 26 s: 21:22:23 → 21:22:49 → 21:23:09 → 21:23:31 → 21:23:52
- Kafka confirma cada mensaje con su partición y offset
- El productor sigue enviando en un loop hasta que se detiene con Ctrl+C

Evidencia 3.3: consumo y gráfica en vivo

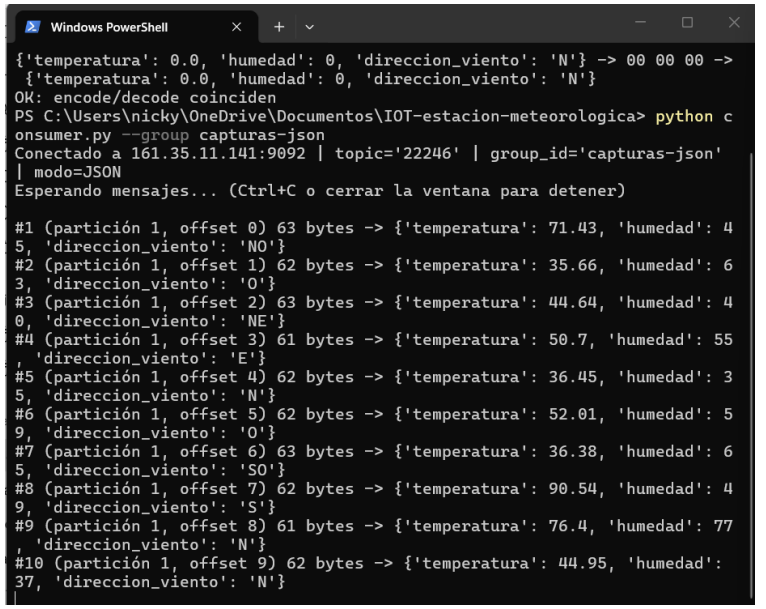
Gráfica en vivo (matplotlib)

Se redibuja con cada mensaje: telemetría pasada (20:41) y entrante (21:22)



python consumer.py

Lee el topic 22246 desde earliest



Kafka y el enfoque Pub/Sub

Ventajas

- Productor y consumidor desacoplados
- Escala con particiones y tolera fallos
- Conserva historial: se puede releer (earliest)

Desventajas

- Requiere operar un broker y configurarlo bien
- Orden garantizado solo dentro de una partición
- Latencia y sobrecarga extra para pocos datos

Tiene sentido para

- Telemetría IoT y flujos de eventos
- Logs, métricas y pipelines de datos
- Microservicios orientados a eventos

No tiene sentido para

- Petición-respuesta síncrona
- Sistemas pequeños con poco tráfico
- Archivos o mensajes muy grandes

Parte 3.4: payload de 3 bytes (24 bits)



¿Cómo cabe la temperatura en 14 bits?

- Se multiplica por 100 y se guarda como entero: 0 a 11000
- 2^14 = 16384 > 11000, así que cabe
- Al decodificar se divide entre 100

Ejemplos verificados

- 56.32 °C, 51 %, SO → 58 01 9B
- 110.00 °C, 100 %, NE → AB E3 27
- encode y decode viven en codec.py, compartido

Evidencia 3.4: encode y decode

codec.py

Compartido por el productor y el consumidor

```python
def encode(data: dict) -> bytes:
    """JSON (dict) -> 3 bytes."""
    t = round(data["temperatura"] * 100)
    h = int(data["humedad"])
    w = DIRECCIONES.index(data["direccion_viento"])

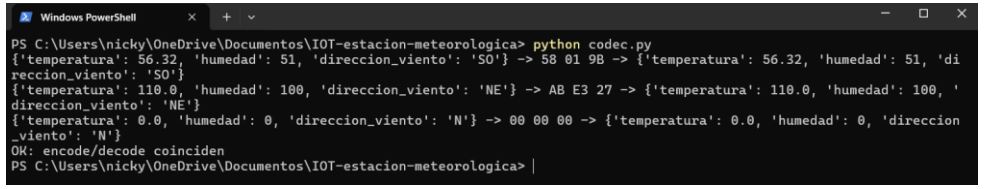
    if not 0 <= t <= 11000:
        raise ValueError(f"Temperatura fuera de rango: {data['temperatura']}")
    if not 0 <= h <= 100:
        raise ValueError(f"Humedad fuera de rango: {h}")

    v = (t << 10) | (h << 3) | w
    return v.to_bytes(3, "big")


def decode(payload: bytes) -> dict:
    """3 bytes -> JSON (dict)."""
    if len(payload) != 3:
        raise ValueError(f"Se esperaban 3 bytes, llegaron {len(payload)}")
    v = int.from_bytes(payload, "big")
    return {
        "temperatura": ((v >> 10) & 0x3FFF) / 100,
        "humedad": (v >> 3) & 0x7F,
        "direccion_viento": DIRECCIONES[v & 0x7],
    }
```

python codec.py

Prueba de ida y vuelta

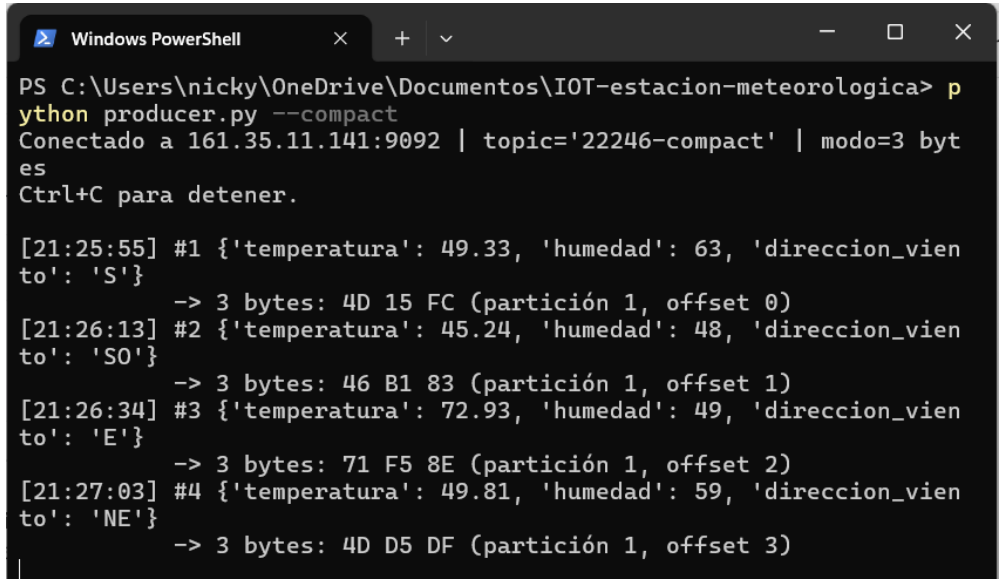


- Los tres casos regresan idénticos después de encode → decode
- Incluye los extremos de cada rango: 110.00 °C, 100 %, NE y 0.00 °C, 0 %, N
- Si un valor está fuera de rango, encode lanza un error en vez de enviar bytes corruptos

Evidencia 3.4: envío y consumo con 3 bytes

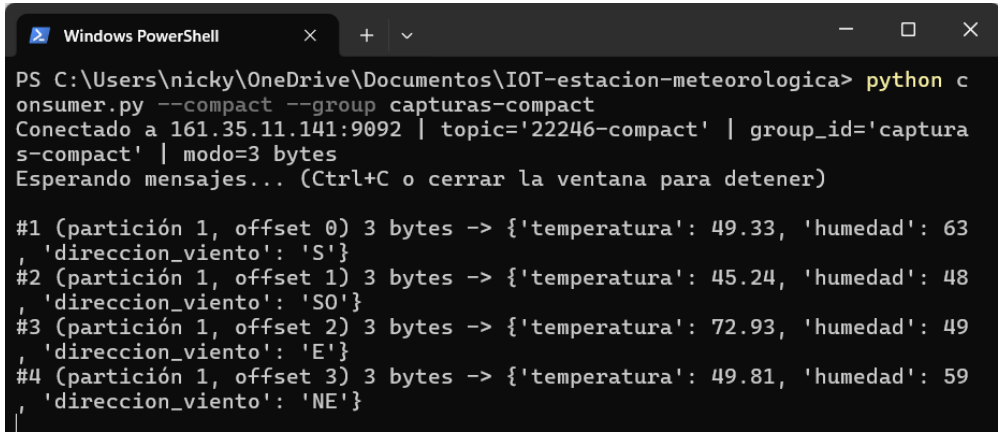
python producer.py --compact

Topic 22246-compact · cada mensaje pesa 3 bytes



python consumer.py --compact

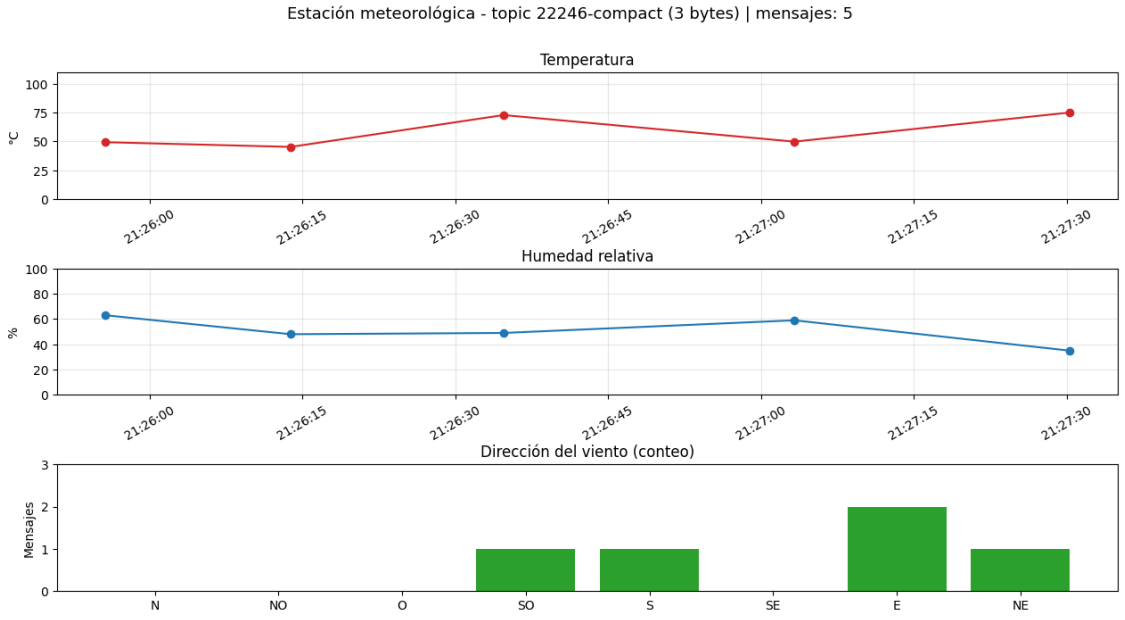
Mismos valores: 4D 15 FC → 49.33 °C, 63 %, S



Evidencia 3.4: gráfica con payload restringido

Gráfica en vivo con 3 bytes

Mismo despliegue que la versión JSON, ahora desde el topic 22246-compact



JSON vs. 3 bytes

62

bytes por mensaje en JSON

3

bytes con el payload restringido

Unas 20 veces menos carga, con los mismos datos y la misma gráfica

Restricciones de payload: análisis

Complejidades

- Encode y decode idénticos en ambos lados
- Se pierde legibilidad y flexibilidad
- Rangos y precisión fijos
- Errores difíciles de depurar

Humedad float con 1 decimal

- 0 a 1000 requiere 10 bits, no 7
- 14 + 10 + 3 = 27 bits, no cabe
- Opción: temperatura a 0.1 °C (11 bits) y 11 + 10 + 3 = 24
- O ampliar el payload o enviar dos mensajes

Ayuda desde Kafka

- compression.type
- linger.ms y batch.size para agrupar
- Serializadores binarios: Protobuf o Avro
- Log compaction por key